COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	Baltazar, Kenji Formilleza, Huone Anthony	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Tree traversals	Tree traversal has 3 different order types; Pre-order, In-order, and Post-order. These traversals use recursion where after checking the last path of its node, the function calls itself to go back to its parent node so that it could traverse to its sibling. Once that part has been searched, recursion will be used again to proceed to another part.
Brute Force	Linear Search	Linear search follows the brute force strategy due to the fact that the main process for linear search is searching every element one by one until it finds its target or until every element in the array has been visited or checked out.
Backtracking	Depth-First Search	Depth-First Search uses blind, exhaustive enumeration to find a solution, it is compatible with the brute-force approach. It is unable to tell if the path its taking is correct or not it instead follows it blindly to the deepest of node until it visited it all, after that it ensures a solution
Greedy	Selection sort	Selection sort fit perfectly on the greedy algorithm because it follows a strict, irreversible strategy of optimizing one at a time, each step ignores the rest of the array to find and swap the absolute minimum value in the current index, once its made the decision the choice is locked and never checked again.
Divide-and-Conquer	Quick Sort	Quick sort's main approach is the divide-and-conquer

		strategy where the algorithm first chooses a pivot and assigns it to its proper position, dividing the array into 2 partitions having both left and right partitions where they are further divided into different sets from another chosen pivot until the remaining arrays have been sorted out.

Table 12-1. *Algorithmic Strategies and Examples*

Screenshot

```cpp
#include <iostream>

int memo[1001];
int getMinSteps(int n);

int main() {
    int n;
        std::cout << "Input 1 to 1000: ";
        std::cin >> n;
        if(n >= 1 && n <= 1000){
            for(int i = 0; i <= n; i++) {
                memo[i] = -1;
            }
            std::cout << "Minimum steps: " << getMinSteps(n) << std::endl;
        }else{
            std::cout << "Invalid input" << std::endl;
        }
    return 0;
}

int getMinSteps(int n) {
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];
    int r = 1 + getMinSteps(n - 1);
    if (n%2 == 0) r = std::min(r, 1 + getMinSteps(n/2));
    if (n%3 == 0) r = std::min(r, 1 + getMinSteps(n/3));
    memo[n] = r;
    return r;
}
```

```
Input 1 to 1000: 100
Minimum steps: 7
```

Analysis

Baltazar	In this section, as seen on the code snippet above, memoization uses recursion. It saves each answer in the memo array so it does not have to solve the same number more than once repeatedly. This is better for those that keeps using the same formula like the fibonacci sequence.
Formilleza	It is a top-down algorithm. Subtraction by one, division by two, and division by three are the three methods it uses to divide n. In the basic case, n = 1 takes zero operations. It looks into the memo table before calculating the solution of a value. If the answer exists already in the memo table, it returns it, otherwise it will be calculated and stored. Because each value is calculated only once, the algorithm complexity is linear and not exponential.

***Table 12-2**. Memoization Implementation*

Screenshot

main2.cpp > getMinSteps(int)

```cpp
#include <iostream>
#include <algorithm>

int getMinSteps(int n);

int main() {
    int n;
    std::cout << "Bottom up" << std::endl;
    do {
        std::cout << "Enter n from 1 to 1000: ";
        std::cin >> n;
        if (n >= 1 && n <= 1000) {
            std::cout << "Minimum steps to reach 1 is " << getMinSteps(n) << std::endl;
        } else if (n != 0) {
            std::cout << "Invalid input" << std::endl;
        }
    } while (n != 0);
    return 0;
}

int getMinSteps(int n) {
    int dp[1001];
    dp[1] = 0;
    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}
```

```
Bottom up
Enter n from 1 to 1000: 100
Minimum steps to reach 1 is 7
Enter n from 1 to 1000: 500
Minimum steps to reach 1 is 9
Enter n from 1 to 1000: 1000
Minimum steps to reach 1 is 9
Enter n from 1 to 1000: 10
Minimum steps to reach 1 is 3
Enter n from 1 to 1000:
```

Analysis

Baltazar	In comparison to memoization, dynamic programming involves iteration which is using loops instead of recursion. As seen on the code snippet, it sets dp[1] = 0, and loops from 2 up to n. So for each i, it starts with 1 + dpt[i-1], then takes a min with 1 + dp[1/ 2] or [1/ 3] when i is divisible by either 2 or 3. And as noticed, the output matches the memoization output, because they share the same time complexity of O(n). But, because it is not recursion, there is no much risk in

		regards to stack overflow, so it is safer for bigger inputs.
	Formilleza	This version begins with the base case, where dp of 1 is equal to 0, and applies a simple loop to populate the array up to n. The order is always guaranteed because every dp value is computed from smaller values that have already been determined. As with memoization, it runs in linear time and requires linear extra space. For large n, it is the more reliable option since it does not rely on recursion, meaning there is no function call overhead and no risk of stack overflow.

Table 12-3. *Bottom-Up Dynamic Programming Implementation*

2. SUPPLEMENTARY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Pseudocode

```
START
  function countPaths with row i, column j and remaining cost
    if remaining cost is less than 0
      return 0
    if i is 0 and j is 0
      if first cell equals remaining cost
        return 1
      else
        return 0
    if i is 0
      return countPaths of i, j minus 1, remaining minus current cell
    if j is 0
      return countPaths of i minus 1, j, remaining minus current cell
    return countPaths of i minus 1, j, remaining minus current cell
      plus countPaths of i, j minus 1, remaining minus current cell
  read the target cost
  print countPaths starting from the last cell
END
```

C++ Code

```cpp
#include <iostream>

const int M = 4;
const int N = 4;
int countPaths(int mtx[M][N], int i, int j, int cost);

int main() {
    int mtx[M][N] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int cost;
    std::cout << "Enter the target cost: ";
    std::cin >> cost;

    int result = countPaths(mtx, M - 1, N - 1, cost);
    std::cout << "Number of paths with cost " << cost << " is " << result << std::endl;

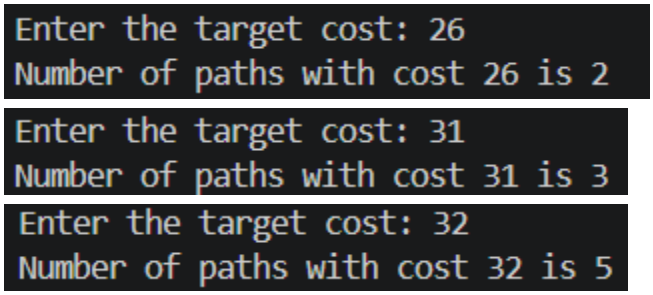
    return 0;
}

int countPaths(int mtx[M][N], int i, int j, int cost){
    if(cost < 0) return 0;
    if(i == 0 && j == 0){
        if(mtx[0][0] == cost){
            return 1;
        }
        return 0;
    }
    if(i == 0){
        return countPaths(mtx, i, j - 1, cost - mtx[i][j]);
    }
    if(j == 0) {
        return countPaths(mtx, i - 1, j, cost - mtx[i][j]);
    }
    return countPaths(mtx, i - 1, j, cost - mtx[i][j]) + countPaths(mtx, i, j - 1, cost - mtx[i][j]);
}
```

Output

```
Enter the target cost: 21
Number of paths with cost 21 is 1
```

```
Enter the target cost: 25
Number of paths with cost 25 is 0
```

	
Analysis	

Baltazar	For this section, the program counts the path from top left to bottom right that adds up to the cost. This uses recursion which starts from the last cell and checking the cell above it and the cell to its left as well. Then it returns 0 when the cost goes below 0 and 1 as well when it reaches the start with the exact cost as well. Though the problem is that it is kinda slow because it uses exponential. Using memoization or dynamic programming would definitely make it faster in comparison.
Formilleza	The algorithm applies recursion and begins at the bottom-right cell. Since you can only move right and down, each cell is a result of the cell on its top or left side. Therefore, the number of paths leading to a cell equals the sum of paths from the top and the left cells with the subtraction of the value of the current cell from the remaining cost. In the base case, the top-left cell is the end of the path, and the remaining cost should be equal to its value to be considered as one path, or else it will be counted as 0. The process is terminated because this path cannot be continued if the remaining cost ever drops below zero. This is a brute force approach with a small amount of pruning all paths are checked, but those that are too expensive are stopped.

3. CONCLUSION

Baltazar	In this activity, I learned about different algorithmic strategies like recursion, brute force, backtracking, greedy, and divide-and-conquer.. Where I learned that recursion is a strategy where a function calls itself like the tree traversals I learned from previous lab activities. Brute force is a strategy where it checks everything until it finds the best possible solution or output, though it is relatively expensive due to the fact that it is checking every single thing. For backtracking, it is just like DFS where after meeting a dead end, it goes back then goes to another node or path. Greedy is the one that I am not familiar with as I just encountered it today, but from what I know, it is basically a non-backtracking strategy where it chooses a path which the output can be optimal or there are cases where the output is not optimal to reach.

	Divide-and-conquer is where it creates subproblems over subproblems just like Quick sort and Merge sorts. Then we have memoization where we use a variable to store a memory and it uses recursion in the code whilst for dynamic programming, it uses iteration like for loops to be able to do a task. In addition, my realization about those two is that both can reach the same output even though their methods are entirely different. Overall, I think I did well in understanding the concepts as well as the codes, though I do need to improve my own understanding in regards to greedy, memoization, as well as dynamic programming.
Formilleza	Overall, we cannot simply take an optimal solution using the option that results in the minimum value for n, for n=10 the greedy algorithm gives fours steps while the minimum is 3, it means that all of the options have to be taken into consideration, then the same values of small n repeats themselves, which allows saving the previous computations to the program more faster. The result of memoization and bottom up approach are almost similar, like for example, 7 steps for n=100, they used the same assumption that the result for n can be achieved through taking into account results for n-1, n/2, and n/3, but the approach differs in how they are implemented, memoization starts with n and then uses recursion, while bottom up iterates from 1 to n which avoids the cost of the function calls, the complexity of both of them in linear, the additional computation has the same logic, the number of paths reach a cell depends on the number of paths to reach the cell above and the cell to the left.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to assist the pseudocode and code in the supplementary. The analysis of the code and the completion of the overall code is still ours.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.